

Data Storage Structures

— Chapter 13

2025 年 11
月

School of Computer Science



北京邮电大学

13.1 Introduction

- **Two issues** in physical DB
 - **data organization**, i.e. **physical storage structure** of data — Ch13
 - **data access**, e.g. index — Ch14
- 解决方案
 - 1. **选择**合适的数据库**物理结构**
 - 2. **设置**合理数据库表**索引**，提高数据查询速度

Outline

- File Organization
- Organization of Records
- Data-Dictionary Storage
- Data Buffer
- Column-oriented Storage

13.2 File Organization

文件组织 / 结构：逻辑和物理

■ 1. 逻辑组织 / 结构

- 流式文件
- 记录文件
- 索引文件

■ 2. 物理组织 / 结构

- 以 block 为单位，存储在 secondary storage, i.e. disk
- 特征：contiguous, linked, indexed

File Organization

- DB is **stored** as a collection of **DB files**
 - 主文件，辅文件，日志文件
 - Each file is a sequence of **records**
 - Each record is in a sequence of **fields**
- **One approach**
 - The record size is **fixed**
 - Each file has records of **one particular type** only
 - **Different** files are used for **different relations**

File Organization

- Each a **relational table** is a set of **tuples**
- Each **file** is a sequence of **records**
 - A **tuple** is stored as a **record** in DB
 - 以**元组**为单位的 DB 逻辑文件在以**记录**为单位的物理文件
- **Represent the tuple as the record in files**
 - **Fixed-length** record
 - **Variable-length** record
 - e.g. *name* varchar (20)

1. Fixed-Length Records

■ Simple approach

- a **block** contains **several records**
- store record i starting from byte $n * (i - 1)$, where n is the size of each record.
- **record access** is simple
but **records** may **cross** blocks
 - modification:
do **not** allow records
to **cross** block **boundaries**

all records are stored in a
contiguous space

contiguous allocation

instructor (ID, name, dept, salary)

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Fixed-Length Records

■ Deletion of record i :

alternatives compacting:

- 1. move records $i + 1, \dots, n$ to $i, \dots, n - 1$
- 2. **move** record n to i
- 3. do **not move** records, but link all **free records** on a **free list**

all records are stored in a contiguous space

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Fixed-Length Records

- Deleting record 3 and compacting

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Fixed-Length Records

- Deleting record 3 and moving last record

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 11	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

Free Lists

- Store **the address** of the first deleted record in the file **header**
- Use this first **deleted** record to **store the address** of the second deleted record, and so on
- Think of these stored **addresses** as **pointers** since they **point** to the location of a record.

□ records can be stored in noncontiguous space

linked allocation

header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

2. Variable-Length Records

- Variable-length records arise in several ways
 - **Record types** allow **variable lengths**, e.g. strings (**varchar**)
 - **Multiple record types** in a file, e.g. Multitable Clustering File
- e.g. **type** `account-list` = **record**
 - `branch-name`: char(22)
 - `account-info`: array[1.. ∞] of
 - record**
 - `account-number` : char(10)
 - `balance` : real
 - end**
- end**

How to store variable-length records in block

- Attributes in the record are stored in order
 - Step1: **fixed length** attributes,
 - Step2: **variable-length** attributes
- ▣ **Variable length attributes** represented by **fixed** size (offset, length), with actual data stored **after** all fixed length attributes
- ▣ Null values represented by null-value bitmap
 - 表示本行某个属性是否取值为 null

instructor: ID(5 varchar) name(10 varchar) dept(10 varchar) salary(8 int)

record 0	10101	Srinivasan	Comp. Sci.	65000
----------	-------	------------	------------	-------

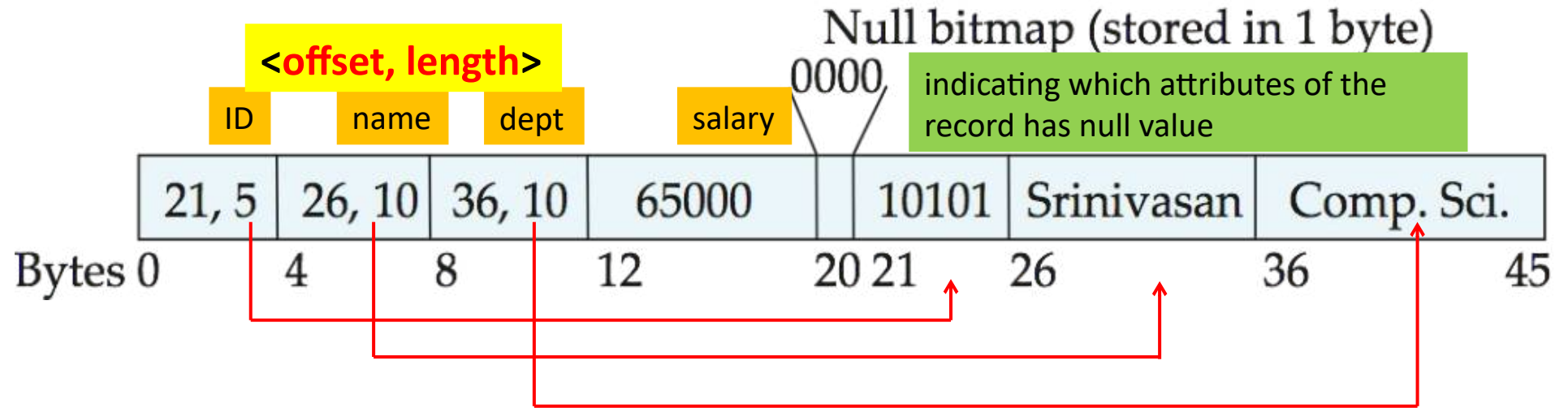


Fig. record 0 is stored in contiguous 46 bytes (0-45)

- Variable length attributes represented by fixed size, with actual data stored after all fixed length attributes

Variable-Length Records: Slotted Page Structure

- The storage space is divided into **fixed-sized slotted pages**, 4KB or 8KB
records are **allocated** to slotted pages

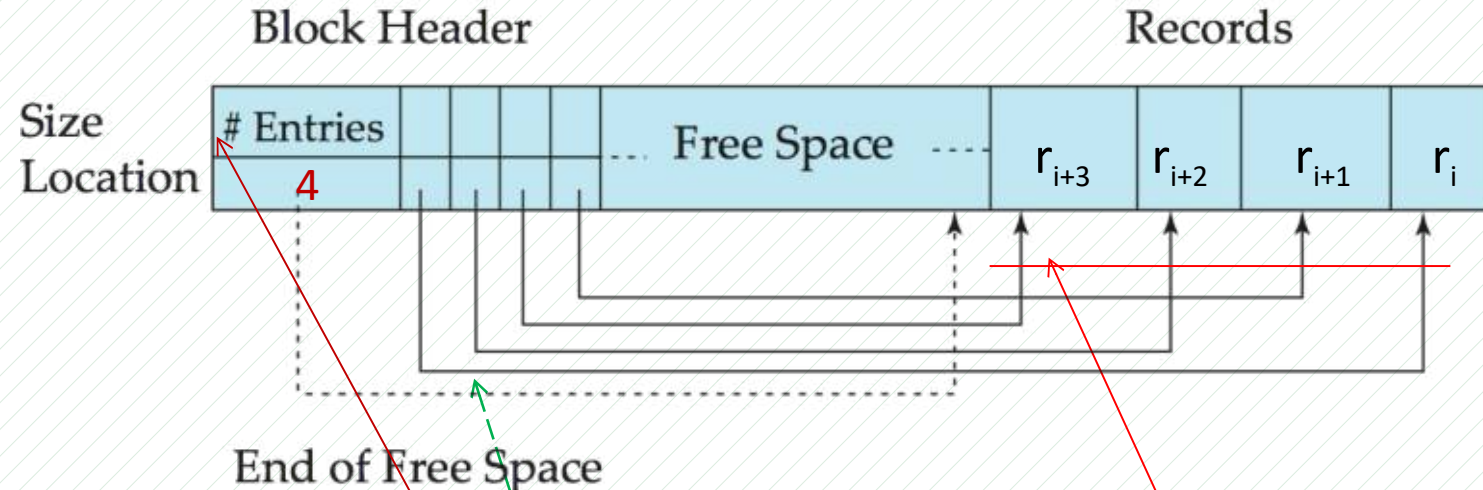


Fig. record_i to record_{i+3} in a page

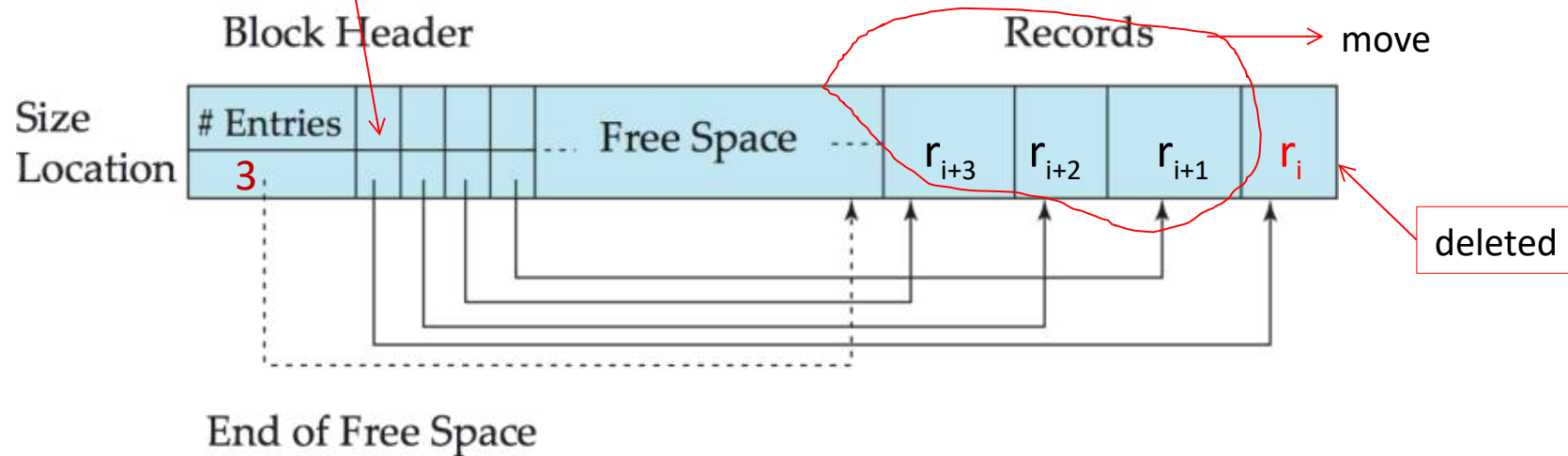
Slotted Page Structure

分槽页结构：

将 DB 文件存储空间在逻辑上
分为多个 page，
每个 page 由多个 slot 组成

- Slotted page** header contains
 - number of record entries, e.g. 4
 - end of free space in the block
 - location and size of each record

- Records can be **moved** around **within a page** to keep **contiguous**
 - r_i is deleted, and r_{i+3} , r_{i+2} , r_{i+1} are moved to the free space created by deletion
 - entry in the header** must be updated, e.g. set to **-1**
- Pointers** should not point directly to record ,
instead they should point to the **entry** for the record in **header**
 - e.g. the pointer to r_i



13.3 Organization of Records in Files

- DB file can be viewed as a set of **records** at **logical level**
- **logical organization** of records determine how to access the records, i.e. access methods
- These records are **logically** organized as
 - **heap, sequential, hash, clustering**
- 文件包括多个记录，记录在**磁盘上**的存放属于文件（记录）结构问题
对于某种结构文件去查找、插入、删除记录，属于文件的存取方法
- 记录的**组织结构**决定了记录的**存取方法**

Organization of Records

- **Heap** – records placed **anywhere** where there is space
 - data has **no primary keys** and **no index**
 - e.g. student (ID, name, total-credits)
- **Sequential** – records stored in **sequential order**
 - based on **the search key** of each record
 - e.g. student (ID, name, total-credits)
student (ID, **name**, total-credits) with an index on **name**
 - Records of each relation may be stored in a separate file
- **Multitable clustering** – records of **different relations** stored in **the same** file
 - motivation: store related records on the same block to **minimize** I/O,
 - e.g. **instructor** **join** **department**,
records of **instructor** and **department** are placed together in a file

1. Heap File Organization

- Any record can be placed anywhere in the file where there is space for the record
- There is **no ordering** of records
- Records usually do not move once allocated
- There is **a single file for a relation**

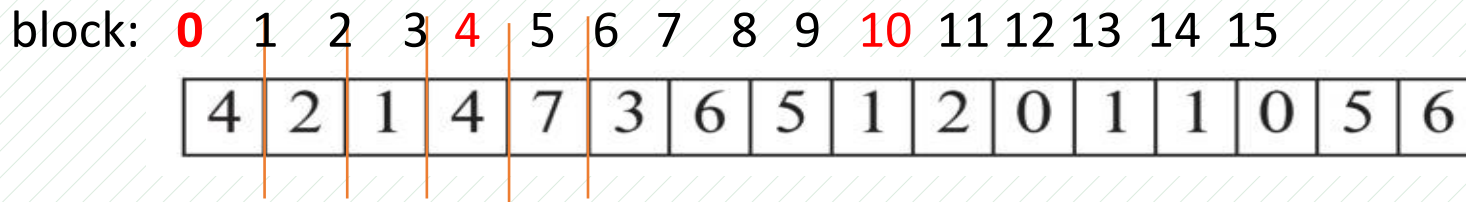
- 以纪录的输入顺序为序，决定了文件中记录顺序
- 纪录的存储顺序与记录中的主键无关
- 创建新关系表：student (ID, name, total-credits) ，
但不在 student 上定义主键、候选键、各类索引，student 被组织为 heap file
 - ▣ 通过 insert，将记录 / 元组加到堆文件中

1. Heap File Organization

■ Free-space map

▣ array with 1 entry per block.

- Each entry records fraction of block that is free
- **Example:** 3 bits (0-7) per block, value divided by 8 indicates fraction of block that is free



block 0: $4/8=50\%$ free

block 4: $7/8$ free

block10: $0/8$, no free

▣ second-level free-space map

- Example: each entry stores maximum 4 entries of first-level free-space map



2. Sequential File Organization

- Records are **logically** ordered by **search-keys**
- Records are stored in sequential order, based on **the search key**
 - 1. records are **physically** stored in search-key order,
 - 2. as **close** to **search-key** order as possible
 - to **minimize** the number of block accesses in sequential processing
- Chain together records **by pointers**
 - The pointer in each record points to the next record in search-key order
 - Sequential access to the file

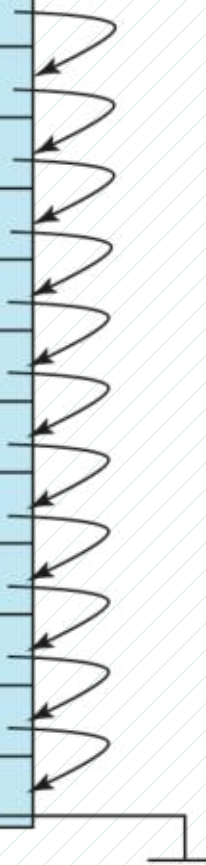
2. Sequential File Organization

- Suitable for sequential processing of the entire file
- ▣ The records in the file are ordered by a search-key

records may be stored
in a noncontiguous
space

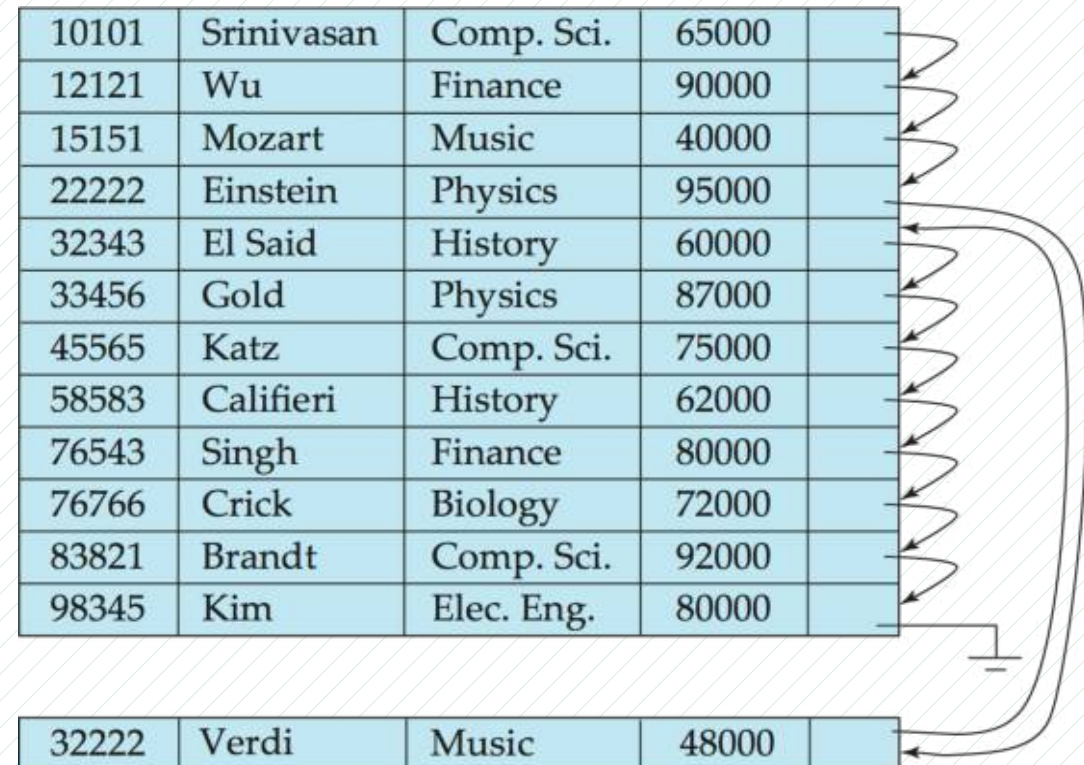
ID

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	



2. Sequential File Organization

- **Deletion** – use pointer chains
- **Insertion** – locate the position where the record is to be inserted
 - ▣ if there is free space insert there
 - ▣ if no free space, insert the record in an **overflow block**
 - ▣ In either case, pointer chain must be updated
- ▣ ★ **Need** to **reorganize** the file from time to time to **restore** sequential order



3. Clustering File Organization

- Store several relations in one file using a **multitable clustering** file organization
 - Records of several different relations are stored in the same file
 - ▣ motivation: store related records on the same block to **minimize I/O**
- 一个 DB 文件可以存储来自多个关系的纪录
 - ▣ 不同关系中有联系的纪录存放在同一 block 中，以提高查找和 I/O 速度

Multitable Clustering File Organization

- 不同关系中有联系的纪录存放在同一 block 中，以提高查找和 I/O 速度

department

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Physics	Watson	70000

instructor

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

multitable clustering
of *department* and
instructor

<u>Comp. Sci.</u>	Taylor	100000
45564	Katz	75000
10101	Srinivasan	65000
83821	Brandt	92000
<u>Physics</u>	Watson	70000
33456	Gold	87000

Multi-table Clustering File Organization

- **Good:** for queries involving *department* ~~instructor~~,
involving one single department and its instructors
- **bad:** for queries involving only *department*
results in variable size records
- Can add pointer chains to link records of a particular relation

Comp. Sci.	Taylor	100000	
45564	Katz	75000	
10101	Srinivasan	65000	
83821	Brandt	92000	
Physics	Watson	70000	
33456	Gold	87000	



4. Hashing File Organization

- The file records are **stored** in **buckets**, the #bucket is the address of the record
- Hash function **on some attributes** (search key) determine the addresses of the records
- **Hash function**
 - ▣ $h : \{\text{search key}\} \rightarrow \{\text{physical address}\}$, i.e. #bucket
 - ▣ the result of the function specifies **in which** block/bucket of the record should be placed
- e.g. Hash file organization of **account (ID, branch-name, balance)**,
 - ▣ **branch-name** as the search key
 - ▣ 表中每个 bucket 对应一个物理文件

bucket 0

bucket 1

bucket 2

bucket 3

A-217	Brighton	750
A-305	Round Hill	350

bucket 4

A-222	Redwood	700

bucket 5

A-102	Perryridge	400
A-201	Perryridge	900
A-218	Perryridge	700

bucket 6

bucket 7

A-215	Mianus	700

bucket 8

A-101	Downtown	500
A-110	Downtown	600

bucket 9

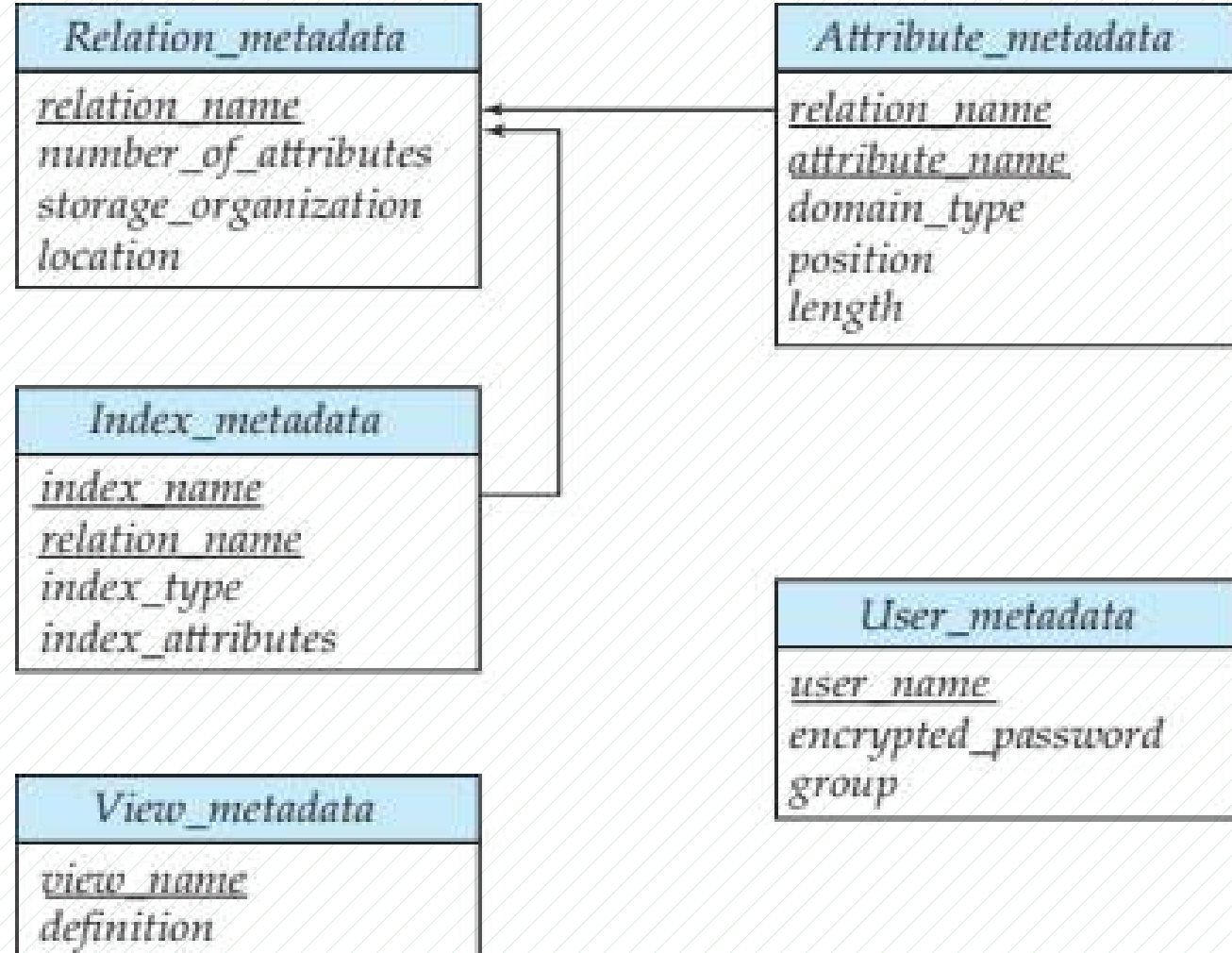
13.4 Data Dictionary Storage

- **Data dictionary** (catalog) stores
- **metadata** : **data about data**, such as
 - names of relations
 - names, types and lengths of attributes
 - names and definitions of views
 - integrity constraints
- **User and accounting information**,
 - passwords
- **Statistical and descriptive data**
 - number of tuples in each relation
- **Physical file organization information**
 - how relation is stored (sequential/hash/...)
 - physical location of relation
- **Information about indices**

Relational Representation of System Metadata

■ Relational representation

- Specialized data structures designed for efficient access in memory



13.5 Data Buffer

- A DB file is partitioned into fixed-length storage units, called **blocks**.
- Blocks are units of both **storage allocation** and **data transfer**.
- DBS seeks to **minimize** the number of **block transfers** between the **disk** and **main memory**.
- We can **reduce** the number of disk accesses by keeping as many blocks as possible in **main memory**

13.5 Data Buffer

- **Def: Buffer**

Portion of main memory available to store **copies** of disk blocks

- **Def: Buffer manager**

allocating buffer space in main memory

- The buffer size is an **adjustable** parameter

It has impact on the system performance, e.g. throughput

data accesses on data item **x** and **y**, e.g. select, insert, delete, update

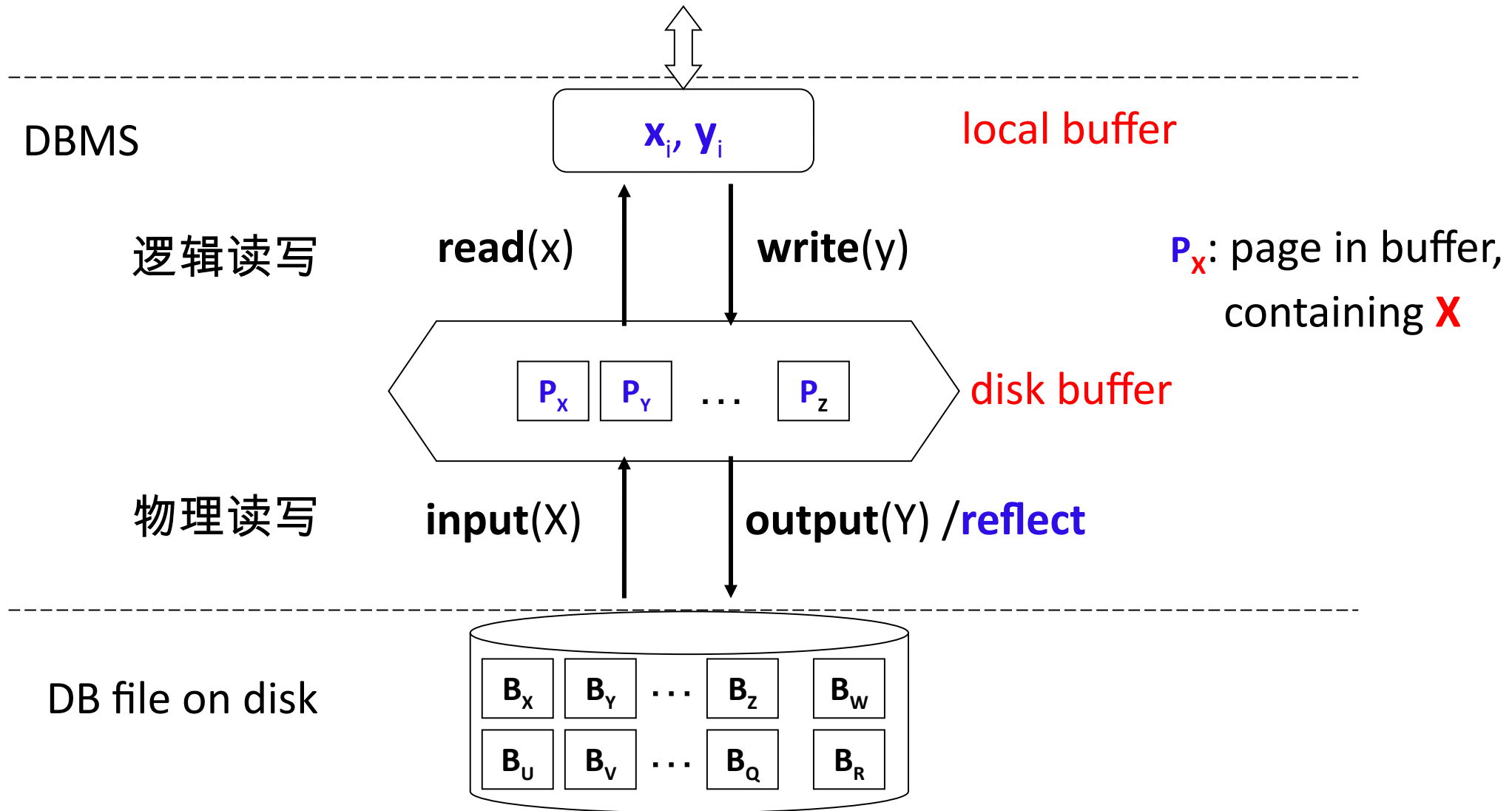


Fig. **read** and **write** operations

Buffer Management

- **Program** (requester) call on **buffer manager** when need block from disk
 - 1. if the block is **already in buffer**, buffer manager returns the address of the block in main memory
 - 2. if the block is **not in buffer**, buffer manager
 - 1. **allocates** space in buffer for the block
 - 1. **Replacing** (throwing out) some other block, if required, to make space for the new block.
 - 2. **Replaced** block written **back** to disk only if it was **modified** since the most recent time that it was written to/fetched from disk.
 - 2. **reads** the block from disk to buffer
returns the address of the block in main memory to **requester**.

Buffer Manager : Buffer replacement strategy

- **Pinned block:** memory block **that is not allowed to be** written back to disk
 - **pin** done before reading/writing data from a block
 - **unpin** done when read /write is complete
- **multiple concurrent pin/unpin operations**
 - Shared and exclusive locks on buffer
- **Concurrent operations** for **shared data**
 - Reading require **shared lock**
 - Updating require **exclusive lock**
 - **Locking rules:**
 - Only one process can get **exclusive lock** at a time
 - **Shared lock cannot** be **concurrently** with **exclusive lock**
 - **Multiple** processes may be given **shared lock** concurrently

Buffer-Replacement Policies

- **Replace** the block **least recently used** (**LRU** strategy)
 - **Idea:** use **past pattern** of block references as **predictor** of future references
 - Use information in user's query to **predict** future references
- **Example:** bad access pattern for LRU:
 - when computing the join of relations **r** and **s** by a nested loops
 - for each tuple **tr** of **r** do
 - for each tuple **ts** of **s** do
 - if the tuples **tr** and **ts** match ...

Buffer-Replacement Policies

- **Toss-immediate strategy:**
 - free the space occupied by block as soon as the final tuple of that block has been processed
- **Most recently used (MRU) strategy:**
 - Pin the block currently being processed.
 - After the final tuple of that block has been processed, the block is unpinned, and it becomes the most recently used block.
- 1. Buffer manager can use **statistical information** regarding the probability that a request will reference a particular relation
 - e.g., **data dictionary** is **frequently** accessed.
 - **Heuristic:** **keep** data-dictionary blocks in main memory buffer
- 2. Buffer manager may reorder **writes**
 - Lead to **corruption** of data structures on disk
 - **Heuristic:** Careful ordering of writes

13.6 Column-Oriented Storage

- **Column-Oriented Storage** known as **columnar representation**

- Store each attribute of a relation separately

- Example

instructor(ID, Name, depart-name, salary)

10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

update instructor
set **salary**=salary + 100
where **ID=15151**

Columnar Representation

■ Benefits:

- ▣ Reduced IO if only some attributes are accessed
- ▣ Improved CPU cache performance
- ▣ Improved compression

update instructor
set salary=salary + 100
where ID=15151

■ Drawbacks:

- ▣ Cost of tuple reconstruction from columnar representation
- ▣ Cost of tuple deletion and update

➤ **Columnar representation** found to be more efficient for decision support than row-oriented representation

➤ **Row-oriented representation** preferable for transaction processing

Appendix 13-1 Data Organization and Access

- Data **organization** in DBS
- Data **access** in DBS
 - ▣ in small DBS
 - ▣ in medium or large scale DBS

Data Organization in DBS

- DBS provides the services of **data organization and access**
- From DBS users, data is **organized** as
 - relational **tables**, **tuples**, and **attributes**
 - user-oriented logical organization of data
- From operating systems, data is stored as **OS file**, known as DB file
 - DB file is **logically organized** as a sequence of **records**
 - DB file resides on **secondary storages**, i.e. disks, in unit of **block**

Data Organization in DBS

- DBS 数据的基本逻辑单位—元组 / 数据行与 OS 文件中数据的基本逻辑单位—文件记录的对应关系
 - ▣ 1 个元组对应于文件系统的 1 个逻辑记录
 - fixed-length record
 - variable-length record
 - ▣ 1 个关系表对应于 1 个 OS 文件，
或 1 个文件存储多个关系表的数据
- record 与磁盘基本存储单位—block 的对应关系关系
 - ▣ in DBS, 1 个 block 包含多个文件逻辑记录

Data Access in DBS

- 小型关系数据库系统（如 dbase、Foxbase），关系表被存储在单独文件，DBMS 利用 OS 文件系统作为 DBMS 物理层实现基础，利用 OS 文件系统实现数据组织与访问
 - **操作系统**为 DB 分配所需的磁盘 block
 - 有可能将逻辑上相邻数据分配到磁盘上不同区域，导致连续访问效率下降
 - **用户**对 DB 文件数据的访问需要借助于 OS 文件系统提供的文件结构和存取路径
 - DBMS 对 OS 依赖太大，不利于数据库系统的移植
 - DBS 数据访问过程，DBMS 的 storage management、OS 文件系统、I/O 子系统完成如下地址变换
 - DB 关系表中元组 / 数据行**到** DB 文件数据的逻辑地址（文件名，记录号）间的映射
 - DB 文件记录号**到**外设上数据物理地址（记录所在的物理块号和具体地址）的映射

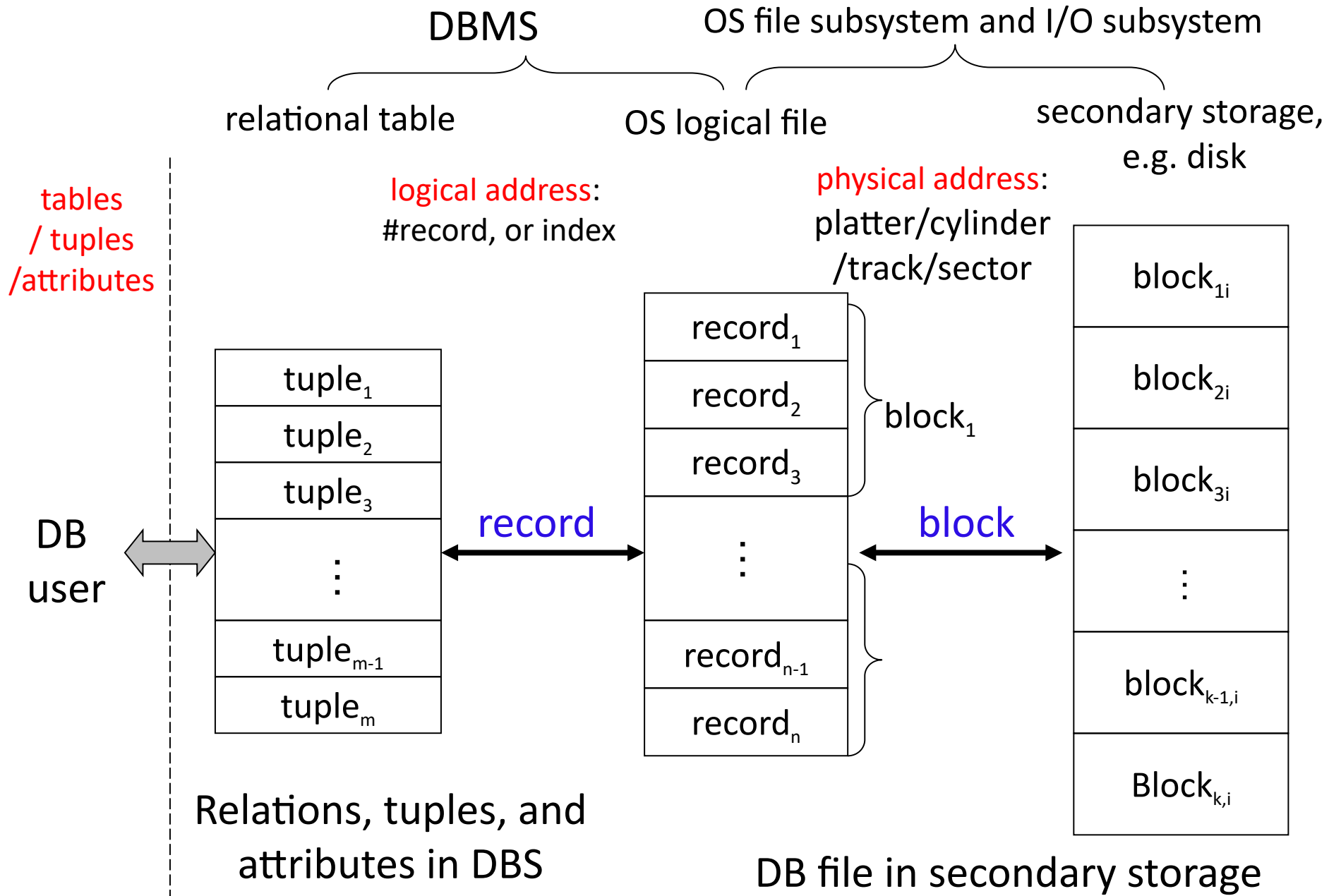


Fig.A.1 Data Access in DBS-I

Data Access in DBS

- 现代大型关系数据库系统，SQL Server、Oracle，为适应数据动态变化、提供数据快速访问路径，DB 数据仍然以 DB 文件的方式存在，但对 DB 文件管理并不直接依赖于 OS 文件系统
- 创建数据库时，DBMS 向 OS 一次性申请所需外存空间（数据库文件），在此外存存储空间中，DBMS 独立地为数据库中的数据设计存储和访问结构，
 - ▣ 分配管理 DB 文件，提供数据访问机制
 - ▣ 根据需要，动态扩充、压缩数据库空间
- SQL Server as Case Study
 - ▣ 借鉴操作系统**虚拟内存管理技术**中 page/frame 的思想



Data Access in DBS

- SQL Server 中，一个数据库由用于存储表 (table) 和索引 (index) 的磁盘 / 外存存储空间构成，这些空间被分配在一个或多个操作系统文件上
 - 主文件，辅文件，日志文件
- 数据库被划分许多逻辑页 (page)，每个逻辑页大小为 8KB
 - 每个数据库文件由多个从 0 开始连续编号的页组成。每个页逻辑上划分为许多从 1 开始连续编号的槽 (slot)，每个槽用于存放一个数据库表中的数据行 / 元组 / 记录
- 数据库文件所在的页可以物理上存储在外存连续区域，如相邻连续的 disk blocks，也可以存储在非连续的 blocks 中

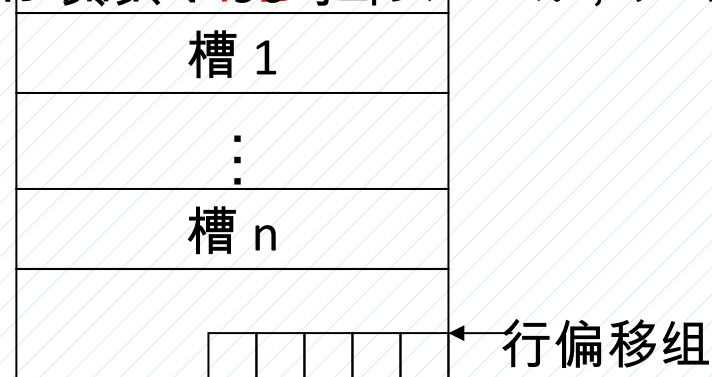


Fig. A.3 Page in SQL Server

Data Access in DBS

- 对于 1 个 DB 表中的数据行 / 元组 / 纪录，SQL Server 用该表所在的数据库文件 ID、数据行所在页 ID 和槽 ID 来标识
 - 数据行的逻辑地址：< 文件 ID, 页 ID, 槽 ID >
- SQL Server 以盘区为单位，为 DB 文件的页分配外存上的物理存储空间
 - 盘区大小为 64K，占用 8 个连续的页
 - DB 文件中数据行 / 元组 / 纪录物理地址：数据行所在盘区的物理地址，如 block#
- SQL Server 利用全局分配映象 GAM 和共享全局分配映象 S GAM 数据结构纪录每个盘区的使用情况
 - 是否空闲、是否共享



Data Access in DBS

- SQL Server 利用**索引分配映象 IAM** 数据结构纪录每个数据库文件的**页**与该页所在物理**盘区**间的对应关系
 - DB 文件中数据对象的**逻辑地址**与**物理地址**映射关系
- SQL Server 利用**索引**实现对关系表中的数据行 / 元组的访问
 - Index: search key $\rightarrow$ < 文件 ID, 页 ID, 槽 ID>
 - SQL Server 可对关系表的**主键**自动建立索引
- SQL Server 数据**存储与访问**机制 (Fig. A.4)
 - 二级地址映射
 - 基于**索引**的 search key $\rightarrow$ 逻辑地址 : < 文件 ID, 页 ID, 槽 ID>
 - 基于 **IAM** 的**数据行逻辑地址** $\diamond$ **数据行物理地址** (盘区物理地址)

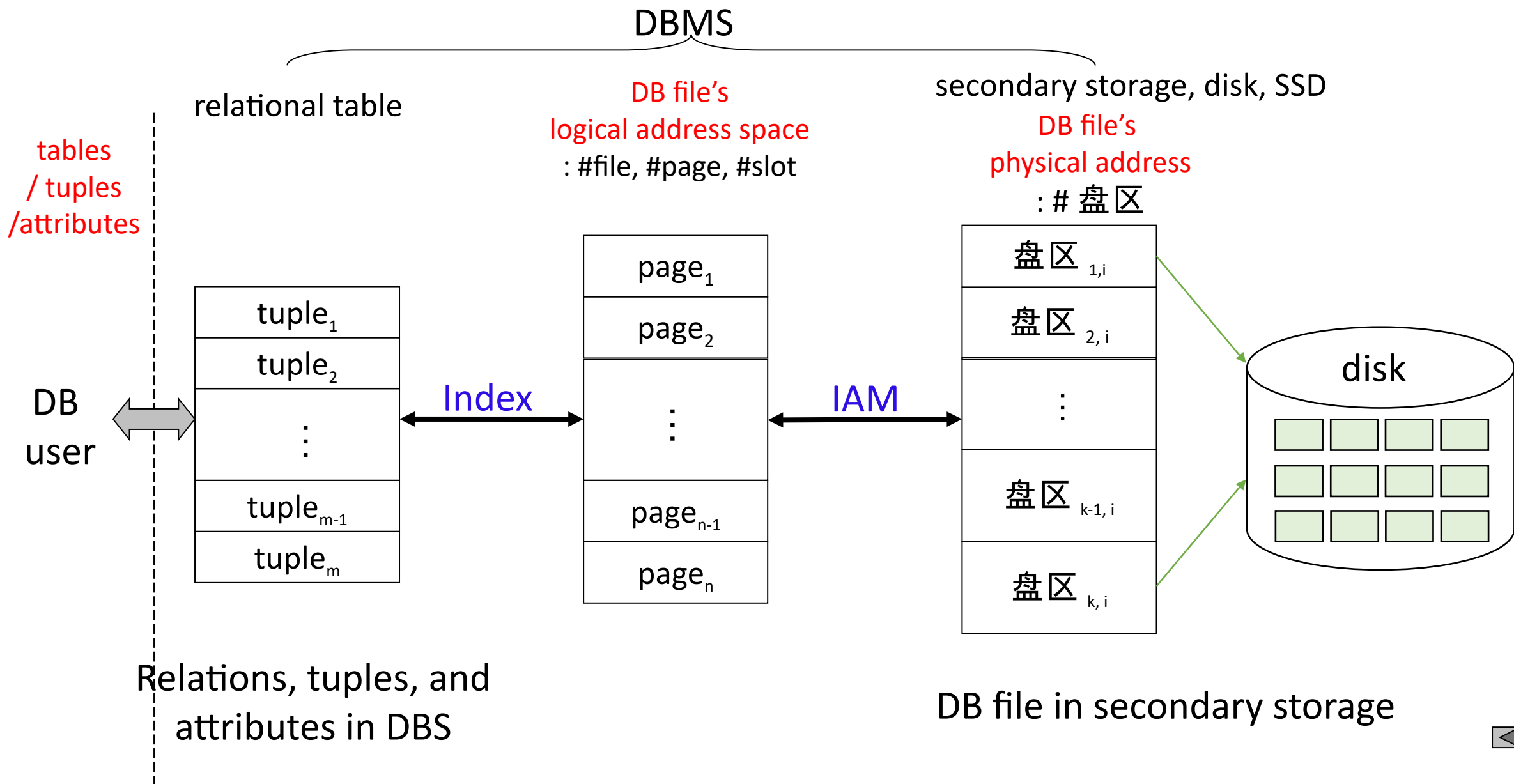


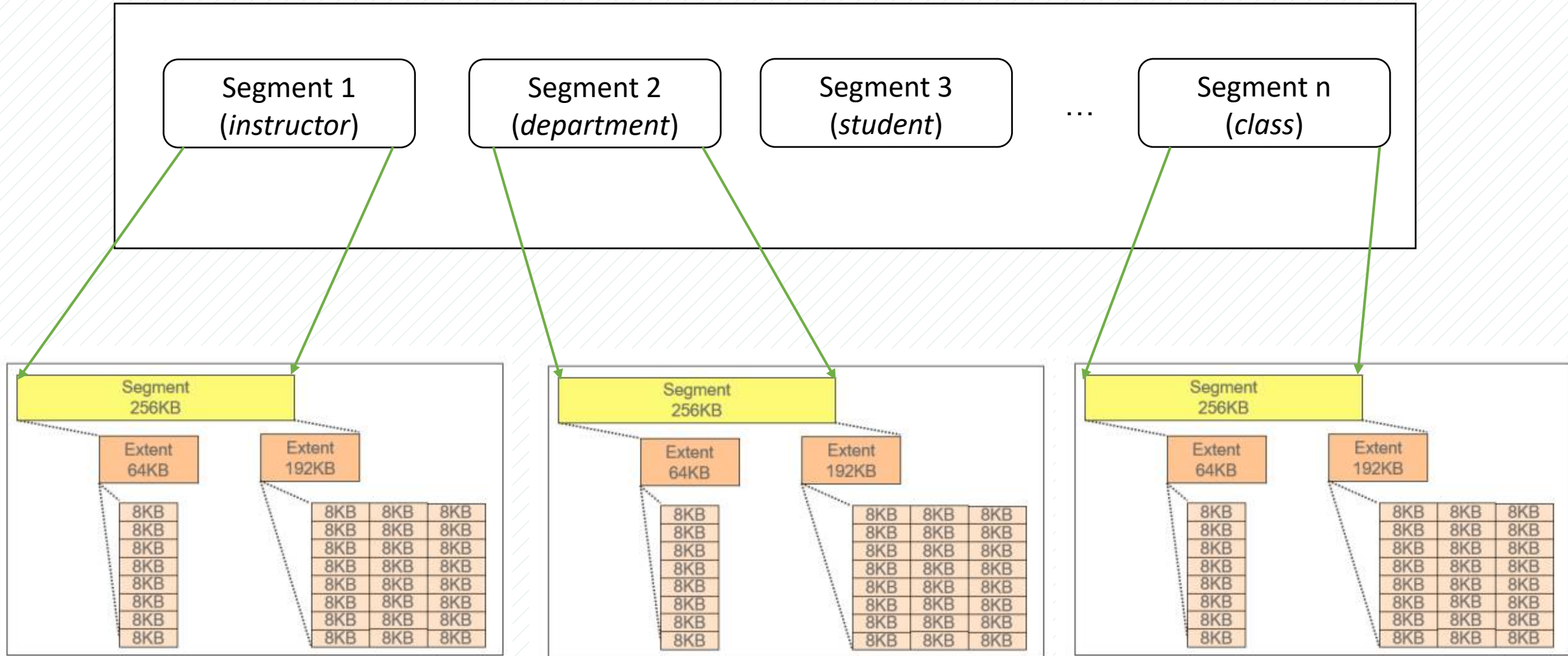
Fig.A.4 Data Access in SQL Server

openGauss 段页式存储管理

- Segmentation with Paging
- 数据库表空间和数据库文件以段—区—页 /block 的逻辑组织方式，进行存储分配和管理
 - 逻辑地址：segment-extent-page/block
- 1 个 database（在一个 tablespace 中）有且仅有一个段空间（segment space），段空间的实际物理存储可以是一个物理文件，也可以拆分存储在多个物理文件上
- 1 个 database 中所有 table 都从该数据库的空间中分配数据，数据库中表的数目和实际物理文件个数无关
 - 避免当数据库中的表比较多时，数据库存储所在的物理文件也比较多的问题
- Database 中的每张 table 有一个逻辑上的 segment，该 table 所有的数据都存在该 segment 上
 - 每个 segment 会挂载多个 extent，每个 extent 是外存上一块连续的物理页，但 1 个 segment 内的各个 extent 可以是不连续的
 - extent 的大小可以根据业务需求调整，避免碎片带来的存储空间浪费

openGauss 段页式存储管理

University database: segment space



openGauss 段页式存储管理

- 段页式文件可以自动扩容，不需要用户手动指定，直到磁盘空间用满或者达到 tablespace 设置的 limit 限制
- 段页式存储不会自动回收磁盘空间
 - 当某些数据表被删除之后，其在段页式文件中占据的空间，会被保留，即段页式文件中会存在一些空洞，磁盘空间没有被释放。
 - 这些空洞会被后面新扩展或者创建出来的表重用
 - 如果不需要重用这些空洞，手动调用系统函数，进行磁盘空间回收，释放磁盘空间
- 物理实现上，每个 segment 对应一个物理文件
 - 例如，每个分区表、每个 hashbucket 表的一个 bucket

openGauss 段页式存储管理

- 四种大小的 extent: 64K , 1M , 8M , 64M
- 段扩展
 - ▣ 当向 table 中添加数据时，可能需要扩展该表对应的 segment ，添加新的 extent
 - ▣ 每一次扩展的 extent 的大小是固定的
 - 前 16 个大小为 64K ，第 17-143 个大小为 1MB
 - 依次类推，具体参数如下表所示

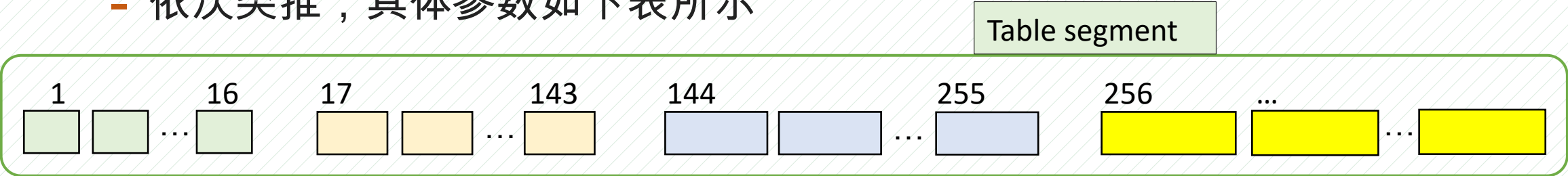


表 1 segment存储extent分类表

Group	Extent size	Extent page count	Extent count range	Total page count	Total size
1	64K 	8	[1, 16]	128	1M
2	1M 	128	[17, 143]	16K	128M
3	8M 	1024	[144, 255]	128K	1G
4	64M 	8192	[256, ...]	...	...

Appendix 13-2 Files Organization in Oracle and OpenGauss

- Oracle 数据库中 7 种数据库表
 - 堆表，索引组织表，簇表
 - heap, sequential, clustering
 - 临时表，压缩表，分区表，嵌套表
- 堆表
 - Oracle 和 openGauss 默认表类型
 - 在数据写入时无需考虑行存放的顺序（按照写入时间先后顺序存放），因此写入速度较高，但由于是无序存放，读取效率较低

■ 索引组织表

- 以索引方式保存表数据，数据根据**主键**顺序进行存储排列，提高**访问速度**
- **缺点**
 - 索引块保存所有的**字段的信息**，需要**更多**的**叶子页面**保存数据，数据量较大的时候会造成访问效率降低
 - 如果**主键**频繁修改，对应的行也就需要**磁盘位置**频繁修改，行需要在**不同的块**之间相互移动
- 使用索引组织表
 - 表的**宽度**（即一行的数据长度）有限
 - 表的**主键**不会或极少更改
 - 表主要用于**查询**，DML 操作较少

Files Organization in Oracle and OpenGauss

- 簇表
 - 簇表也称为 Cluster，在没有数据表和索引时候，Cluster 段是可以单独存在
 - 依据一定规则，如连接键（Join Key），可将多个数据表数据保存在同一个段中。并且依据一定场景实现快速检索连接
 - 在 openGauss 中，无法兼容此功能

